Introduction to arduino Evive and pictoblox

Presented By C.K.Vinodh

CEN

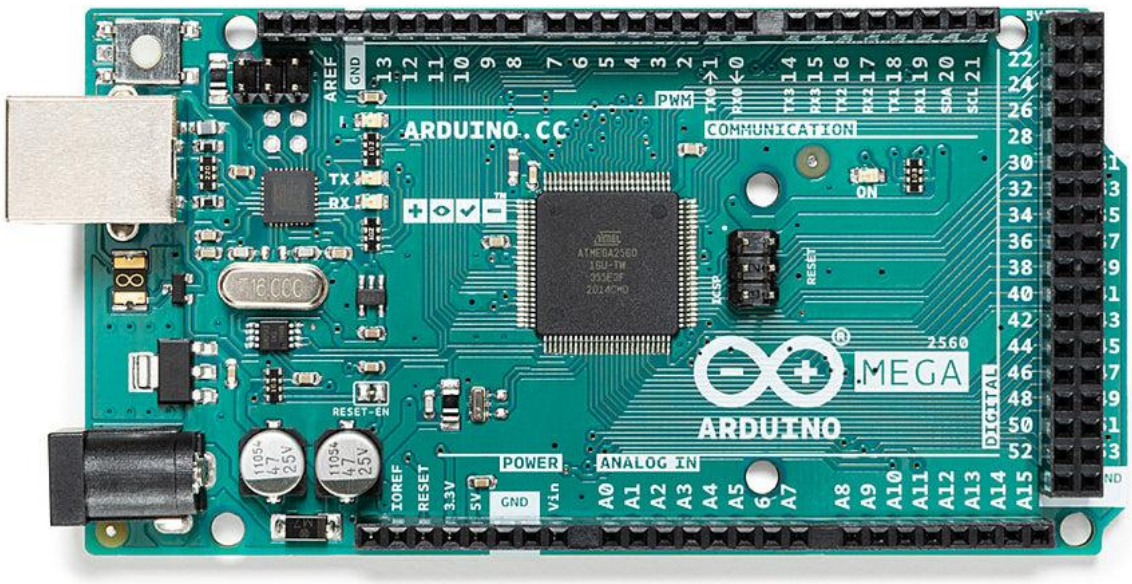
(Centre for Computational Engineering and Networking)

Amrita Vishwa Vidyapeetham

Ettimadai,Coimbatore,India

S.NO	CONTENT	Page
1	Arduino overview	3
2	Arduino mega pinout	6
3	Power	8
4	Pin maping	9
5	Ardunio mega analog pin sampling rate.	14
6	Resolution	15
7	PWM	16
8	Pictoblox	19
9	Evive	23
10	Evive IOT kit	37
11	sensors	45
12	Arduino blink	46
13	Arduino fade	50
13	Example code for library function	52
14	Evive blink	55
15	Evive Dable and pictoblox light on off	56
17	Evive pwm	65
18	Basic of Electrical and Electronics	69

ARDUINO



Overview

The **Arduino Mega 2560** is a microcontroller board based on the ATmega2560. It has 54 digital input/output pins (of which 15 can be used as PWM outputs), 16 analog inputs, 4 UARTs (hardware serial ports), a 16 MHz crystal oscillator, a USB connection, a power jack, an ICSP header, and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started. The Mega 2560 board is compatible with most shields designed for the Uno and the former boards Duemilanove or Diecimila.

The Mega 2560 is an update to the Arduino Mega, which it replaces.

Getting Started

Find inspiration for your projects with the Mega 2560 board from our tutorial platform Project Hub.

You can find in the Getting Started with Arduino MEGA2560 Rev 3 section all the information you need to configure your board, use the Arduino Software (IDE), and start tinkering with coding and electronics.

From the Tutorials section you can find examples from libraries and built-in sketches as well other useful information to expand your knowledge of the Arduino hardware and software.

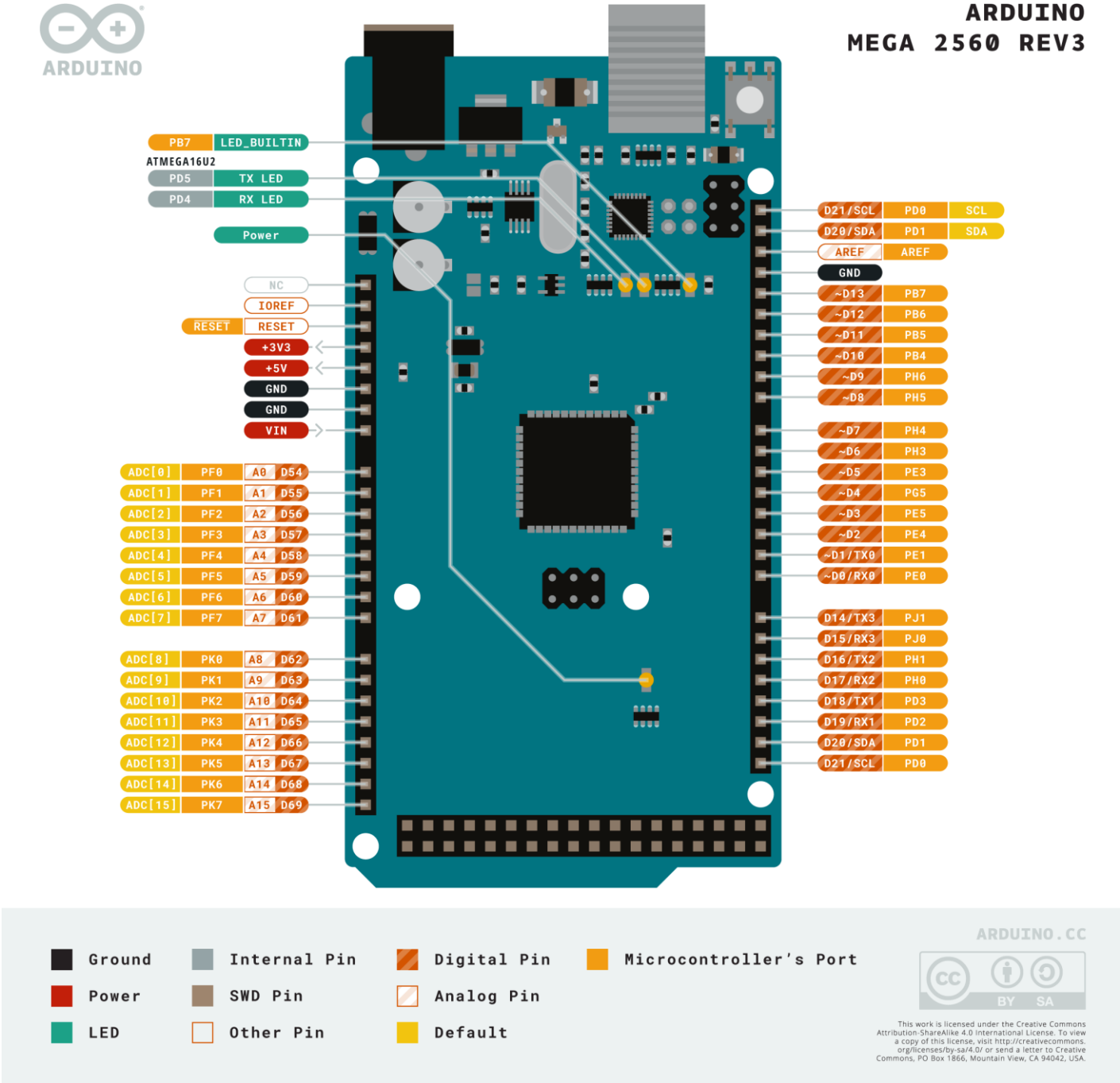
Need Help?

Check the Arduino Forum for questions about the Arduino Language, or how to make your own Projects with Arduino. Need any help with your board please get in touch with the official Arduino User Support as explained in our Contact Us page.

Tech specs

MICROCONTROLLER	ATmega2560
OPERATING VOLTAGE	5V
INPUT VOLTAGE (RECOMMENDED)	7-12V
INPUT VOLTAGE (LIMIT)	6-20V
DIGITAL I/O PINS	54 (of which 15 provide PWM output)
ANALOG INPUT PINS	16
DC CURRENT PER I/O PIN	20 mA
DC CURRENT FOR 3.3V PIN	50 mA
FLASH MEMORY	256 KB of which 8 KB used by bootloader
SRAM	8 KB
EEPROM	4 KB
CLOCK SPEED	16 MHz
LED_BUILTIN	13
LENGTH	101.52 mm
WIDTH	53.3 mm
WEIGHT	37 g

Pinout Diagram



Download the full pinout diagram as PDF here.

Learn more

- **ARDUINO DOCS**For the full technical documentation, tutorials and much more, visit Arduino Docs

FAQs

Programming

The Mega 2560 board can be programmed with the Arduino Software (IDE). For details, see the reference and tutorials.

The ATmega2560 on the Mega 2560 comes preprogrammed with a bootloader that allows you to upload new code to it without the use of an external hardware programmer. It communicates using the original STK500 protocol (reference, C header files).

You can also bypass the bootloader and program the microcontroller through the ICSP (In-Circuit Serial Programming) header using Arduino ISP or similar; see these instructions for details.

The ATmega16U2 (or 8U2 in the rev1 and rev2 boards) firmware source code is available in theArduino repository. The ATmega16U2/8U2 is loaded with a DFU bootloader, which can be activated by:

- On Rev1 boards: connecting the solder jumper on the back of the board (near the map of Italy) and then resetting the 8U2.
- On Rev2 or later boards: there is a resistor that pulling the 8U2/16U2 HWB line to ground, making it easier to put into DFU mode. You can then use Atmel's FLIP software (Windows) or the DFU programmer (Mac OS X and Linux) to load a new firmware. Or you can use the ISP header with an external programmer (overwriting the DFU bootloader). See this user-contributed tutorial for more information.

Warnings

The Mega 2560 has a resettable polyfuse that protects your computer's USB ports from shorts and overcurrent. Although most computers provide their own internal protection, the fuse provides an extra layer of protection. If more than 500 mA is applied to the USB port, the fuse will automatically break the connection until the short or overload is removed.

Power

The Mega 2560 can be powered via the USB connection or with an external power supply. The power source is selected automatically.

External (non-USB) power can come either from an AC-to-DC adapter (wall-wart) or battery. The adapter can be connected by plugging a 2.1mm center-positive plug into the board's power jack. Leads from a battery can be inserted in the GND and Vin pin headers of the POWER connector.

The board can operate on an external supply of 6 to 20 volts. If supplied with less than 7V, however, the 5V pin may supply less than five volts and the board may become unstable. If using more than 12V, the voltage regulator may overheat and damage the board. The recommended range is 7 to 12 volts.

The power pins are as follows:

- Vin. The input voltage to the board when it's using an external power source (as opposed to 5 volts from the USB connection or other regulated power source). You can supply voltage through this pin, or, if supplying voltage via the power jack, access it through this pin.

- 5V. This pin outputs a regulated 5V from the regulator on the board. The board can be supplied with power either from the DC power jack (7 - 12V), the USB connector (5V), or the VIN pin of the board (7-12V). Supplying voltage via the 5V or 3.3V pins bypasses the regulator, and can damage your board. We don't advise it.

- 3V3. A 3.3 volt supply generated by the on-board regulator. Maximum current draw is 50 mA.

- GND. Ground pins.

- IOREF. This pin on the board provides the voltage reference with which the microcontroller operates. A properly configured shield can read the IOREF pin voltage and select the appropriate power source or enable voltage translators on the outputs for working with the 5V or 3.3V.

Memory

The ATmega2560 has 256 KB of flash memory for storing code (of which 8 KB is used for the bootloader), 8 KB of SRAM and 4 KB of EEPROM (which can be read and written with the EEPROM library).

Input and Output

See the mapping between Arduino pins and Atmega2560 ports:

PIN MAPPING ATmega2560

Each of the 54 digital pins on the Mega can be used as an input or output, using pinMode(),digitalWrite(), and digitalRead() functions. They operate at 5 volts. **Each pin can provide or receive 20 mA as recommended** operating condition and has an internal pull-up resistor (disconnected by default) of 20-50 k ohm. A maximum of 40mA is the value that must not be exceeded to avoid permanent damage to the microcontroller.

In addition, some pins have specialized functions:

- Serial: 0 (RX) and 1 (TX); Serial 1: 19 (RX) and 18 (TX); Serial 2: 17 (RX) and 16 (TX); Serial 3: 15 (RX) and 14 (TX). Used to receive (RX) and transmit (TX) TTL serial data. Pins 0 and 1 are also connected to the corresponding pins of the ATmega16U2 USB-to-TTL Serial chip.

- External Interrupts: 2 (interrupt 0), 3 (interrupt 1), 18 (interrupt 5), 19 (interrupt 4), 20 (interrupt 3), and 21 (interrupt 2). These pins can be configured to trigger an interrupt on a low level, a rising or falling edge, or a change in level. See the attachInterrupt() function for details.

- PWM: 2 to 13 and 44 to 46. Provide 8-bit PWM output with the analogWrite() function.

- SPI: 50 (MISO), 51 (MOSI), 52 (SCK), 53 (SS). These pins support SPI communication using theSPI library. The SPI pins are also broken out on the ICSP header, which is physically compatible with the Arduino /Genuino Uno and the old Duemilanove and Diecimila Arduino boards.

- LED: 13. There is a built-in LED connected to digital pin 13. When the pin is HIGH value, the LED is on, when the pin is LOW, it's off.

- TWI: 20 (SDA) and 21 (SCL). Support TWI communication using the Wire library. Note that these pins are not in the same location as the TWI pins on the old Duemilanove or Diecimila Arduino boards.

See also the mapping Arduino Mega 2560 PIN diagram.

The Mega 2560 has 16 analog inputs, each of which provide 10 bits of resolution (i.e. 1024 different values). By default they measure from ground to 5 volts, though is it possible to change the upper end of their range using the AREF pin and analogReference() function. There are a couple of other pins on the board:

- AREF. Reference voltage for the analog inputs. Used with analogReference().
- Reset. Bring this line LOW to reset the microcontroller. Typically used to add a reset button to shields which block the one on the board.

Communication

The Mega 2560 board has a number of facilities for communicating with a computer, another board, or other microcontrollers. The ATmega2560 provides four hardware UARTs for TTL (5V) serial communication. An ATmega16U2 (ATmega 8U2 on the revision 1 and revision 2 boards) on the board channels one of these over USB and provides a virtual com port to software on the computer (Windows machines will need a .inf file, but OSX and Linux machines will recognize the board as a COM port automatically. The Arduino Software (IDE) includes a serial monitor which allows simple textual data to be sent to and from the board. The RX and TX LEDs on the board will flash when data is being transmitted via the ATmega8U2/ATmega16U2 chip and USB connection to the computer (but not for serial communication on pins 0 and 1).

A SoftwareSerial library allows for serial communication on any of the Mega 2560's digital pins.

The Mega 2560 also supports TWI and SPI communication. The Arduino Software (IDE) includes a Wire library to simplify use of the TWI bus; see the documentation for details. For SPI communication, use the SPI library.

Physical Characteristics and Shield Compatibility

The maximum length and width of the Mega 2560 PCB are 4 and 2.1 inches respectively, with the USB connector and power jack extending beyond the former dimension. Three screw holes allow the board to be attached to a surface or case. Note that the distance between digital pins 7 and 8 is 160 mil (0.16"), not an even multiple of the 100 mil spacing of the other pins.

The Mega 2560 is designed to be compatible with most shields designed for the Uno and the older Diecimila or Duemilanove Arduino boards. Digital pins 0 to 13 (and the adjacent AREF and GND pins), analog inputs 0 to 5, the power header, and ICSP header are all in equivalent locations. Furthermore, the main UART (serial port) is located on the same pins (0 and 1), as are external interrupts 0 and 1 (pins 2 and 3 respectively). SPI is available through the ICSP header on both the Mega 2560 and Duemilanove / Diecimila boards. Please note that I2C is not located on the same pins on the Mega 2560 board (20 and 21) as the Duemilanove / Diecimila boards (analog inputs 4 and 5).

Automatic (Software) Reset

Rather then requiring a physical press of the reset button before an upload, the Mega 2560 is designed in a way that allows it to be reset by software running on a connected computer. One of the hardware flow control lines (DTR) of the ATmega8U2 is connected to the reset line of the ATmega2560 via a 100 nanofarad capacitor. When this line is asserted (taken low), the reset line drops long enough to reset the chip. The Arduino Software (IDE) uses this capability to allow you to upload code by simply pressing the upload button in the Arduino environment. This means that the bootloader can have a shorter timeout, as the lowering of DTR can be well-coordinated with the start of the upload.

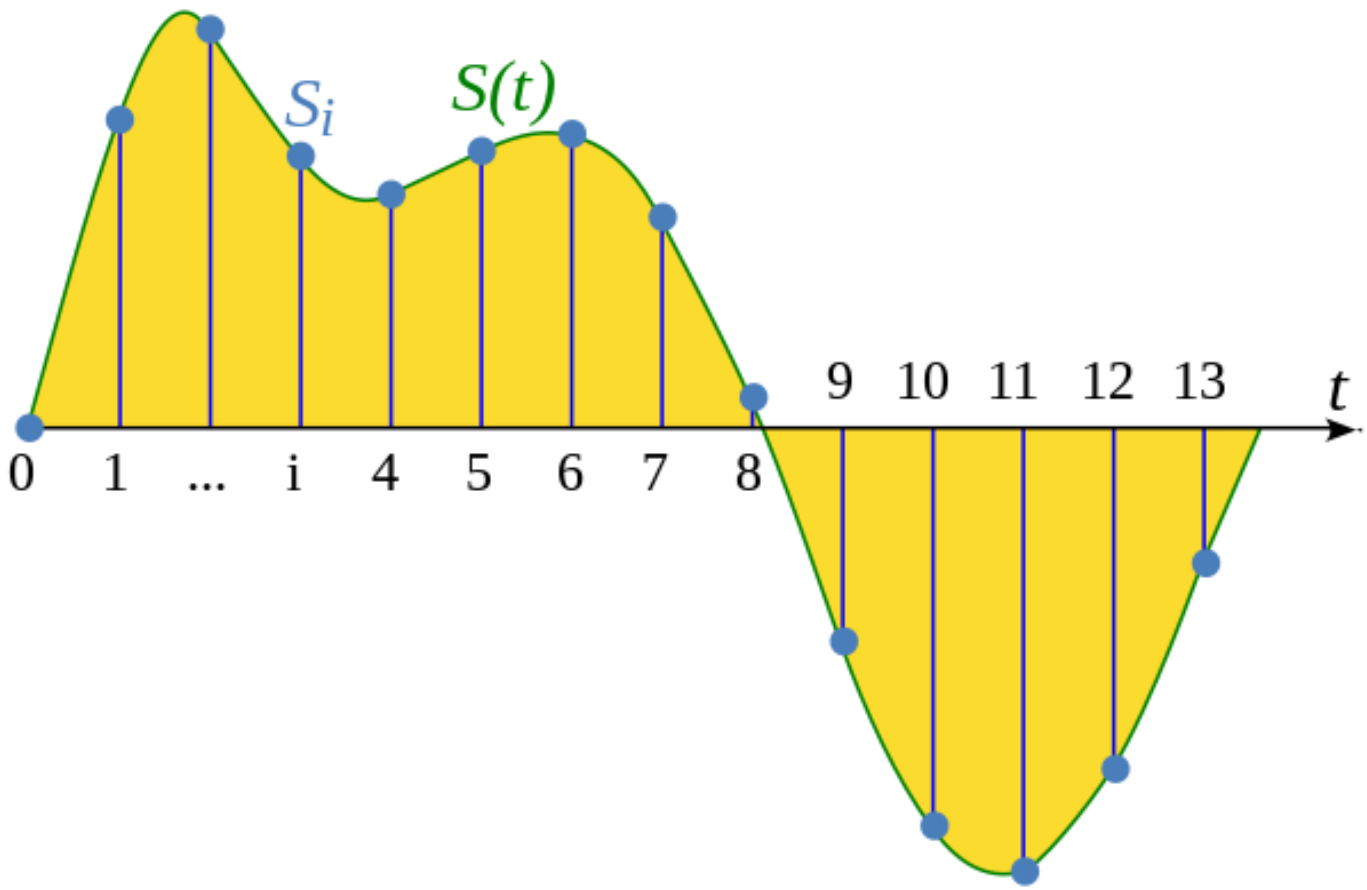
This setup has other implications. When the Mega 2560 board is connected to either a computer running Mac OS X or Linux, it resets each time a connection is made to it from software (via USB). For the following half-second or so, the bootloader is running on the ATMega2560. While it is programmed to ignore malformed data (i.e. anything besides an upload of new code), it will intercept the first few bytes of data sent to the board after a connection is opened. If a sketch running on the board receives one-time configuration or other data when it first starts, make sure that the software with which it communicates waits a second after opening the connection and before sending this data.

The Mega 2560 board contains a trace that can be cut to disable the auto-reset. The pads on either side of the trace can be soldered together to re-enable it. It's labeled "RESET-EN". You may also be able to disable the auto-reset by connecting a 110 ohm resistor from 5V to the reset line; see this forum thread for details.

Ardunio mega analog pin sampling rate.

For a 16 MHz Arduino the ADC clock is set to 16 MHz/128 = 125 KHz. Each conversion in AVR takes 13 ADC clocks so 125 KHz /13 = 9615 Hz. That is the maximum possible sampling rate, but the actual sampling rate in your application **depends on the interval between successive conversions calls**.

The **sampling frequency** or **sampling rate**, f_s, is the average number of samples obtained in one second, thus $f_s = 1/T$. Its units are **samples per second** or hertz e.g. 48 kHz is 48,000 samples per second.



What is resolution?

Resolution in this context refers to the conversion of an analog voltage to a digital value in a computer (and vice versa). A computer is a digital machine and thus stores a number as a series of ones and zeroes. If you are storing a digital 2-bit number you can store 4 different values: 00, 01, 10, or 11. Now, say you have a device which converts an analog voltage between 0 and 10 volts into a 2-bit digital value for storage in a computer. This device will give digital values as follows:

Voltage	2-Bit Digital Representation
0 to 2.5	00
2.5 to 5	01
5 to 7.5	10
7.5 to 10	11

So in this example, the 2-bit digital value can represent 4 different numbers, and the voltage input range of 0 to 10 volts is divided into 4 pieces giving a voltage resolution of 2.5 volts per bit. A 3-bit digital value can represent 8 (2^3) different numbers. A 12-bit digital value can represent 4096 (2^{12}) different numbers. A 16-bit digital value can represent 65536 (2^{16}) different numbers. It might occur to you at this point that a digital input could be thought of as a 1-bit analog to digital converter. Low voltages give a 0 and high voltages give a 1.

analogRead()

[Analog I/O]

Description

Reads the value from the specified analog pin. Arduino boards contain a multichannel, 10-bit analog to digital converter. This means that it will map input voltages between 0 and the operating voltage(5V or 3.3V) into integer values between 0 and 1023. On an Arduino UNO, for example, this yields a resolution between readings of: 5 volts / 1024 units or, 0.0049 volts (4.9 mV) per unit. See the table below for the usable pins, operating voltage and maximum resolution for some Arduino boards.

The input range can be changed using analogReference(), while the resolution can be changed (only for Zero, Due and MKR boards) using analogReadResolution().

On ATmega based boards (UNO, Nano, Mini, Mega), it takes about 100 microseconds (0.0001 s) to read an analog input, so the maximum reading rate is about 10,000 times a second.

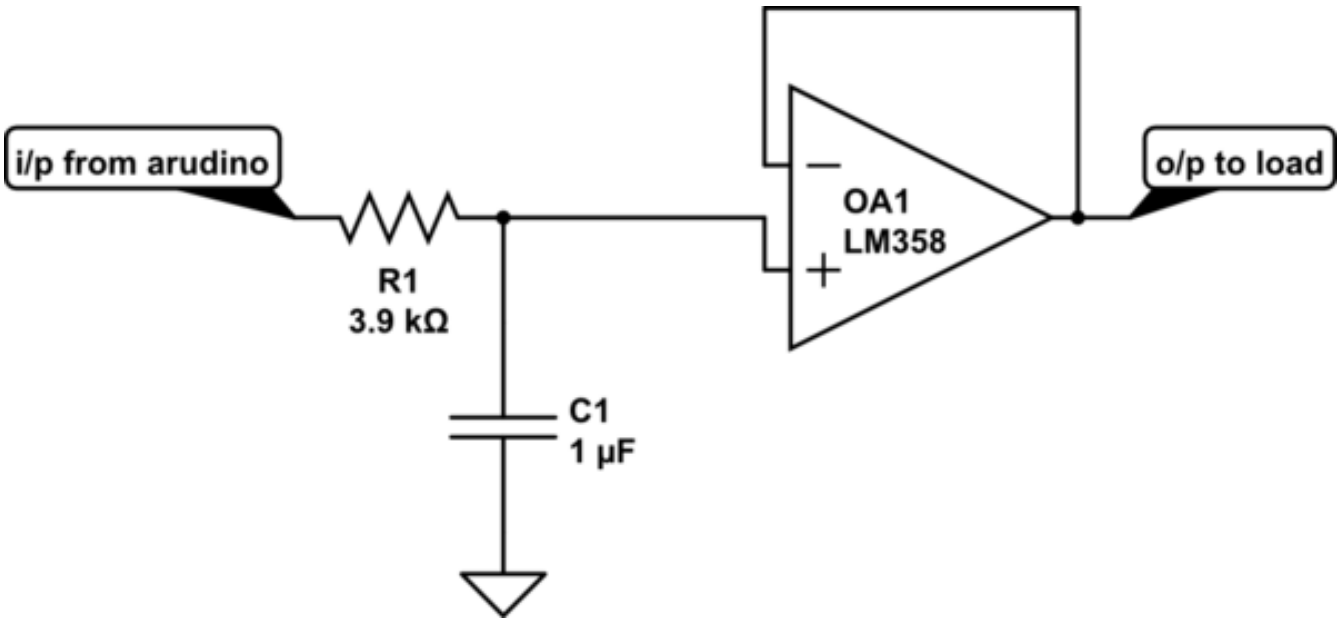
BOARD	OPERATING VOLTAGE	USABLE PINS	MAX RESOLUTION
Uno	5 Volts	A0 to A5	10 bits
Mini, Nano	5 Volts	A0 to A7	10 bits
Mega,Mega2560, MegaADK	5 Volts	A0 to A14	10 bits
Micro	5 Volts	A0 to A11*	10 bits
Leonardo	5 Volts	A0 to A11*	10 bits
Zero	3.3 Volts	A0 to A5	12 bits**
Due	3.3 Volts	A0 to A11	12 bits**
MKR Family boards	3.3 Volts	A0 to A6	12 bits**

*A0 through A5 are labelled on the board, A6 through A11 are respectively available on pins 4, 6, 8, 9, 10, and 12
**The default analogRead() resolution for these boards is 10 bits, for compatibility. You need to use analogReadResolution() to change it to 12 bits.

PWM

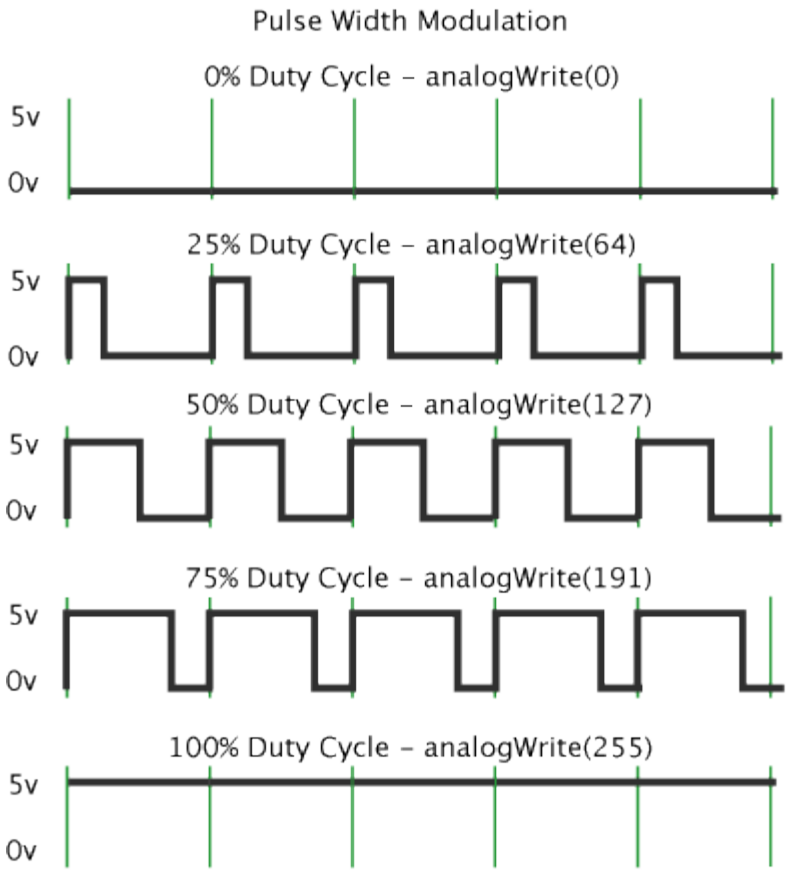
The Fading example demonstrates the use of analog output (PWM) to fade an LED. It is available in the File->Sketchbook->Examples->Analog menu of the Arduino software.

Pulse Width Modulation, or PWM, is a technique for getting analog results with digital means. Digital control is used to create a square wave, a signal switched between on and off. This on-off pattern can simulate voltages in between the full Vcc of the board (e.g.,5V on Uno, 3.3 V on a MKR board) and off (0 Volts) by changing the portion of the time the signal spends on versus the time that the signal spends off. The duration of "on time" is called the pulse width. To get varying analog values, you change, or modulate, that pulse width. If you repeat this on-off pattern fast enough with an LED for example, the result is as if the signal is a steady voltage between 0 and Vcc controlling the brightness of the LED.



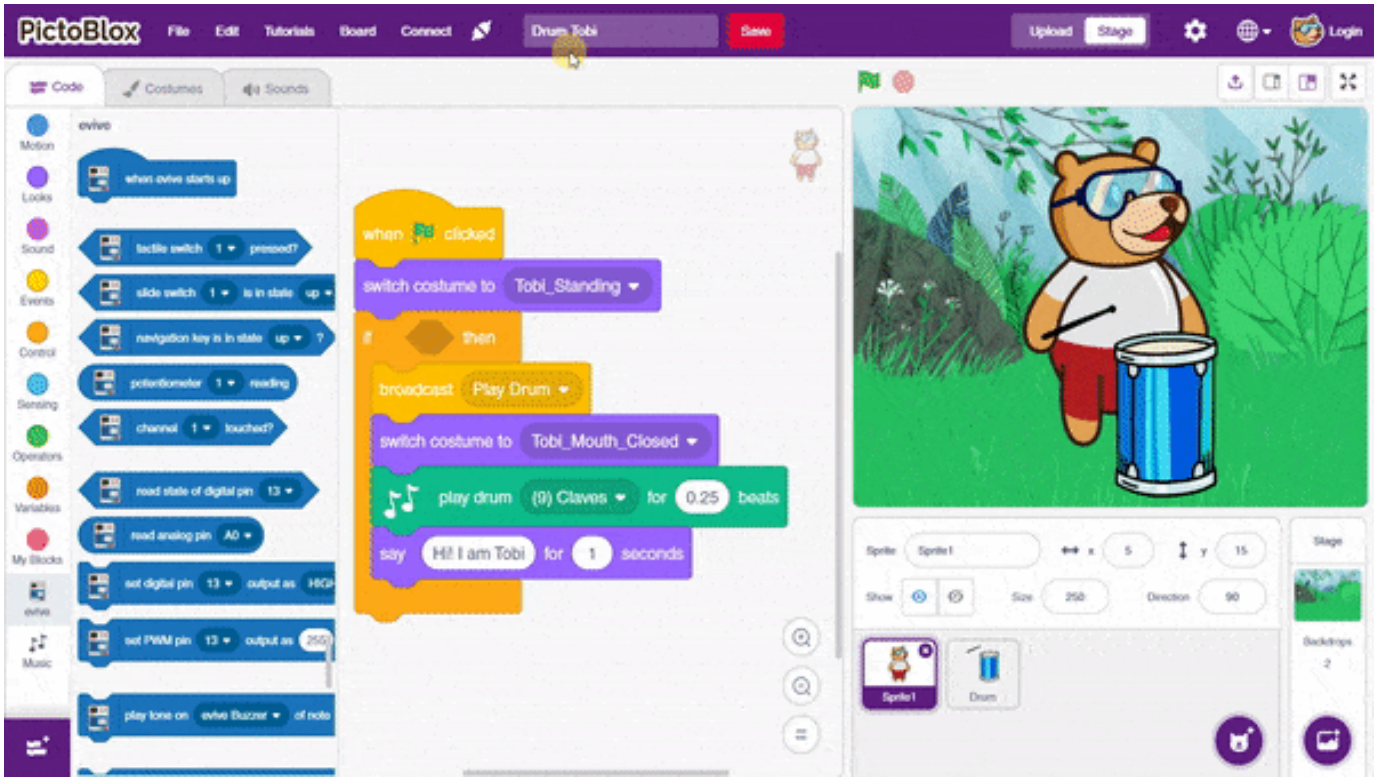
PWM

In the graphic below, the green lines represent a regular time period. This duration or period is the inverse of the PWM frequency. In other words, with Arduino's PWM frequency at about 500Hz, the green lines would measure 2 milliseconds each. A call to analogWrite() is on a scale of 0 - 255, such that analogWrite(255) requests a 100% duty cycle (always on), and analogWrite(127) is a 50% duty cycle (on half the time) for example.



Once you get this example running, grab your arduino and shake it back and forth. What you are doing here is essentially mapping time across the space. To our eyes, the movement blurs each LED blink into a line. As the LED fades in and out, those little lines will grow and shrink in length. Now you are seeing the pulse width.

Pictoblox

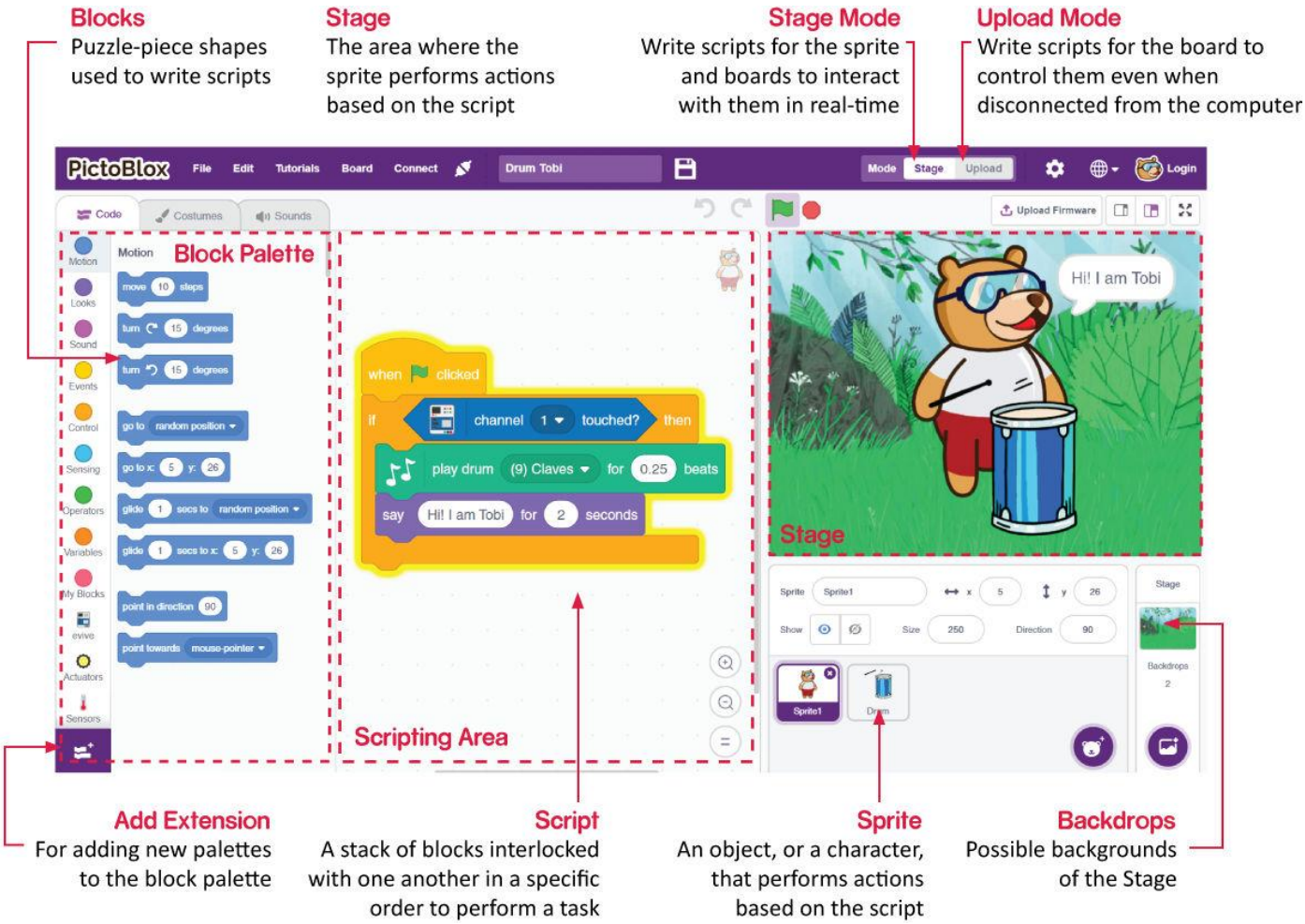


Installing

https://thestempedia.com/tutorials/getting-started-pictoblox

Getting started

https://thestempedia.com/tutorials/getting-started-pictoblox



PictoBlox, like Scratch, has the following basic elements:

1. Sprites
2. Stage
3. Blocks
4. Script

Sprites

A sprite is an object, or a character, which performs different actions in the projects. It understands and obeys the instructions given to it. The bear you see in the image is a sprite. His name is Tobi. He will be your coding buddy in PictoBlox!

A sprite has **costumes**, which are one of the many appearances of the sprite. Some sprites have multiple costumes, and you'll use **blocks** to perform actions.

Stage

The stage is the area where the sprite performs actions according to your program. The stage has its own scripts and sounds. You can decorate the stage using images; these images are called **backdrops**. You can choose a backdrop from PictoBlox's inbuilt library, paint one yourself, upload an image from your computer or even click a picture.

Script

A script is a program or a code in PictoBlox/Scratch lingo. It is a set of 'blocks' that are arranged below one another in a specific order to perform a task or a series of tasks. You can write multiple scripts all of which can then run simultaneously. You can write the scripts only in the scripting area which is in the center of the screen.

Blocks

A block is like a jigsaw puzzle piece that is used to write programs by simply dragging and dropping them below one another in the scripting area. Using blocks to write codes eliminates the errors that might occur while writing codes in traditional programming languages.

Block Palette

The block palette is under the Code tab. It consists of different palettes such as Motion, Sound, and Control. Each palette has different blocks that perform functions specified by the palette name. E.g., the blocks in the Motion palette will control the motion of the sprite and the blocks in the Control palette will control the working of the script depending on specific conditions.

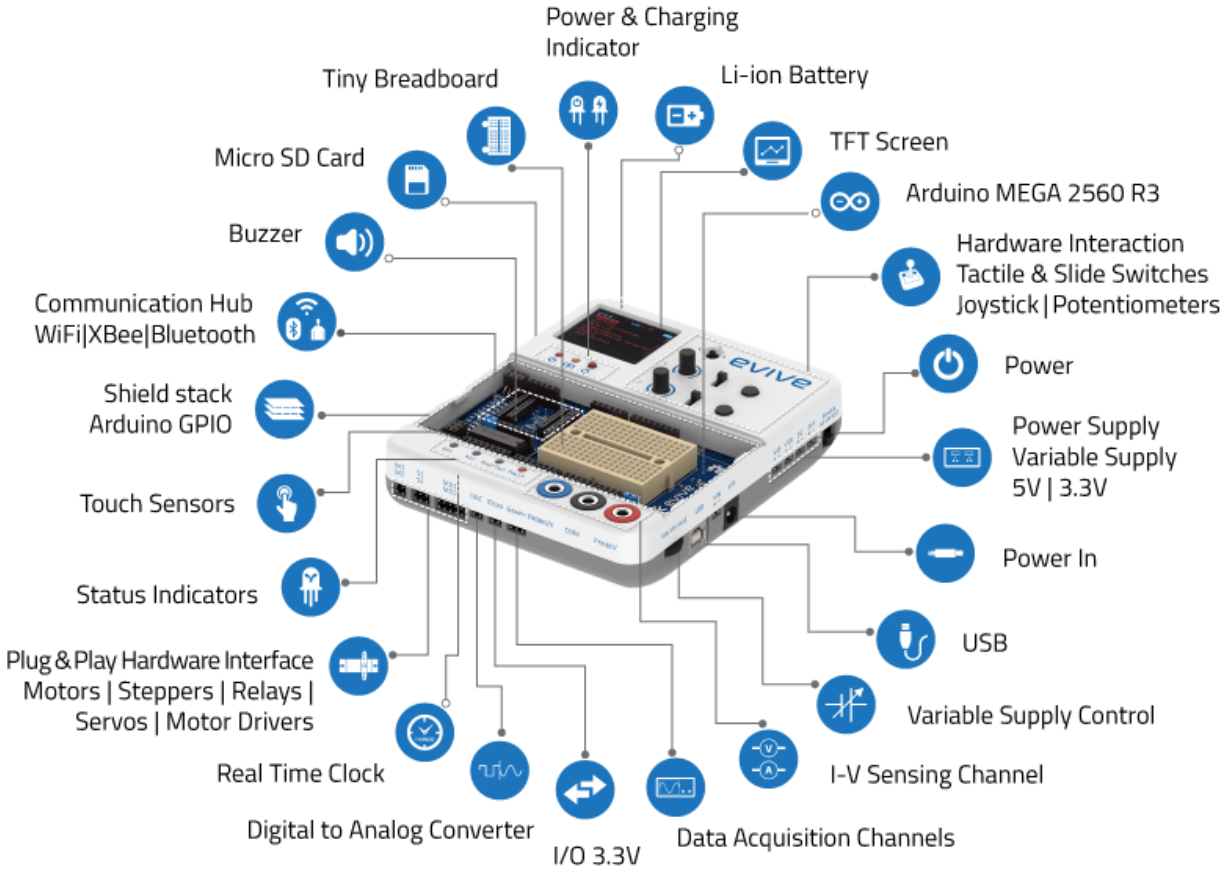
There are other varieties of block palettes that can be loaded from the Add Extension button located at the left bottom.

Modes

PictoBlox has two modes:

1. **Stage Mode**: In this mode, you can write scripts for the sprite and boards like evive to interact with sprites in real-time. If you disconnect the board with Pictoblox, you cannot interact anymore.
2. **Upload Mode**: This mode allows you to write scripts and upload it to the board so that you can use even when it is not connected to your computer, for example, you need to upload a script for making moving robots.

EVIVE



Features

Power Module

evive has its own inbuilt Li-ion 2200mAh battery, giving it portability and durability. It can also be powered externally by a USB wall charger, power bank, DC wall adapter, laptops and >12V batteries. These inputs have reverse polarity, overvoltage and overcurrent protection to protect you and your evive in the event of any mistakes. evive also has a host of power output ports, with a fixed 5V/3A supply, fixed 3.3V/0.8A supply and a tunable voltage supply from 1.25 – 30V/3A.

Power Switch

Internal Battery

Power Input

5V Output

Variable Voltage Output

LED Indicators

Reverse Polarity Protection

Overcurrent Protection

Overvoltage Protection

Hardware Interaction

With two knob potentiometers, 2 tactile switches, 2 slide switches, a 5-way navigation key and 12 capacitive touch inputs, evive makes it incredibly easy to interact with your projects without any messy circuits.

Knob Potentiometer

Tactile Switches

Slide Switches

5-way Navigation Key

12 Capacitive Touch Inputs

Plug and Play Hardware Interface

Direct outputs for running DC motors, stepper motors, pneumatics, relays and servos (for low current consuming hardware approx. 1A) make it as easy as plugging in your device into evive and directly playing with it using the hardware interaction module. For controlling high current consuming devices, it has 2 ports with signal outputs to control your motor drivers.

DC Motor

DC Motor Driver

Servo Motor

Stepper Motor

Relay

Pnuematics

Modes of Feedback

evive has multi-modal feedback, both visual and auditory. The 1.8” TFT screen with 18-bit colour and a resolution of 160x128px displays the menu

which controls evive, but can be programmed according to your custom needs. It has 1 single-colour and 2 dual-colour programmable LEDs, along with fixed LEDs for serial communication, power indication and battery charging status. The auditory mode consists of a 2k-10kHz piezo buzzer.

Piezo Buzzer

TFT Screen

Programmable LEDs

Data Aquisition Module

evive has a precise data acquisition module, with 2 24-bit ADCs capable of voltage sensing (voltmeter) up to +/- 30V with 10mV accuracy, and current sensing (ammeter) up to +/- 3A with 3mA accuracy, at a maximum sensing frequency of 75 kHz. With the on-board real-time clock, it can log data with timestamps, and store it in an SD card or transmit it via serial or Bluetooth.

Real Time Clock

Ammeter (-3A to +3A)

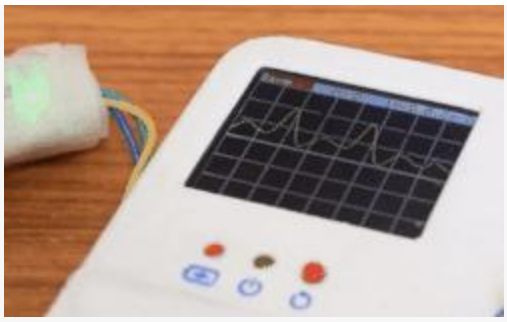
Voltmeter (-5V to +5V)

Voltmeter (-30V to +30V)

2 Channels

24-bit ADC

SD Card Adapter



Mini Oscilloscope

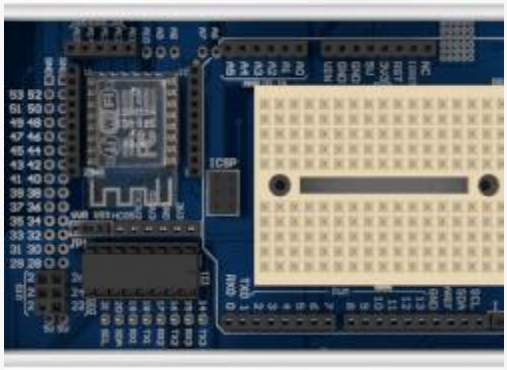
evive can be used as mini oscilloscope to visualize voltage and current measurements. Is has lots of function like scaling voltage level, time scaling to see the reading in better way, trigger to start the measurement at some set value, scanning modes like single or continuous, offset, save readings to SD card and send data to computer.

2 Channels

Time Scaling

Upto 1kHz Frequency

Send Data over USB



Internet of Things

The IoT hub contains ports for the ESP-8266, HC-05 and XBee modules, which provides WiFi, Bluetooth and RF connectivity. The modules can be plugged in directly and used.

Bluetooth (HC05)

Wi-Fi ESP8266)

XBee

Menu Interface (evive Firmware)

evive offers a novel way of interacting with your hardware and code, via the menu-interface displayed on the TFT screen and controlled using the 5-way switch. The pre-defined menu functionality includes driving motors and servos, data sensing and mini-oscilloscope, serial monitor, data logging, function generator and pin states. Users can also add their own functions easily and access them through the user defined functions submenu.

Multiple User Programs

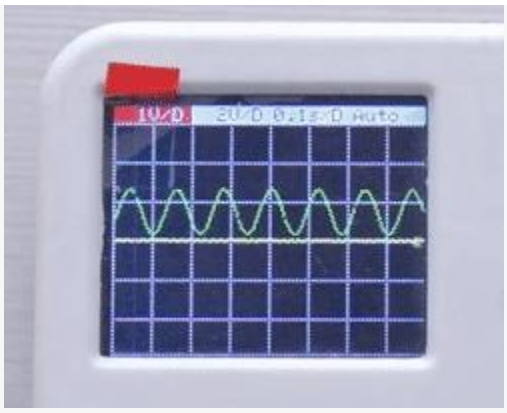
TFT Serial Monitor

Status Indicator

Plug and Play Controls

Current and Voltage Sensing

Pin State Monitor



Beta Function Generator

evive is also a mini-function generator with a 12-bit DAC and an IIC interface. The pre-defined menu offers generation of sine, square, triangular and sawtooth waves upto 530 Hz and 2.5V amplitude.

Sine Wave

Triangular Wave

Square Wave

Saw-tooth Wave

Arduino Core Interface

The Magic Lid gives access to the entire Arduino Uno pinout which can be used for Uno shields. 28 Digital I/O, 10 Analog In, 15 PWM and 6 interrupt pins are accessible through the Magic Lid. The MEGA's SPI, IIC and TTL interfaces are also available. evive also has two 3.3V digital I/O for your 3.3V CMOS logic level devices.

Digital I/O Pins

Analog Input Pins

PWM Pins

Interrupt Pins

Serial Pins

3.3V CMOS

Uno Sheild Stack

IIC

SPI

Physical

evive is compact, light-weight and drop-safe. VESA-compatible 75mm mounting holes are provided to mount evive wherever you need. Although

evive has a mini-breadboard inside its magic lid for small circuits, you can attach your regular-size breadboards to evive by the breadboard mounts provided.

Size

Weight

Mount Holes

Cooling Vents

Breadborad Mount

What you get with evive IoT Kit?

1 X evive (without internal battery)

1 x 3.7V 2600mAh Lithium Ion Battery

1 x Flathead Screwdriver

5 x M3 Bolts of 12mm length (VESA Mounting)

2 x Light Dependent Resistors

1 x DC to Terminal Block

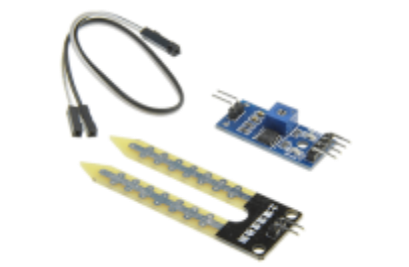
1 x HC05 Bluetooth Module

1 x ESP8266 Wi-Fi Module

4 x 5V Relay

1 x PIR Motion Sensor

1 x Temperature Sensor DHT11



1 x Soil Moisture Sensor

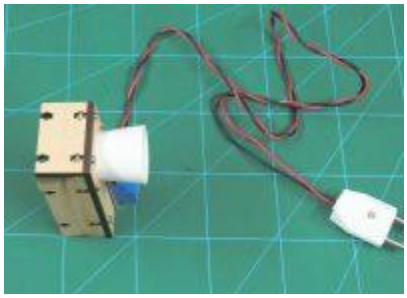
1m x Feeder Pipe

2 x Drip Emitter

2 x T Connector

2 x Emitter Stakes

1 x Teflon Tape



1 x Bulb Holder Assembly

1 x Water Pump 12V 3 W

1 x 12V 2A Power Adapter

1 x EU-US Converter (Provided on Request)

EU-AU Converter (Provided on Request)

Jumper Cables: 25 X Male to Male Jumper Cable | 25 X Male to Female Jumper Cable | 25 X Female to Female Jumper Cable

Multiple Programming Platforms

Compatible with several of hobbyists' favorite programming platforms like the Arduino IDE, PictoBlox, MATLAB, and Python among others.

SENSORS

sensor is a device that produces an output signal for the purpose of sensing a physical phenomenon.

In the broadest definition, a sensor is a device, module, machine, or subsystem that detects events or changes in its environment and sends the information to other electronics, frequently a computer processor. Sensors are always used with other electronics.

A sensor's sensitivity indicates how much its output changes when the input quantity it measures changes. For instance, if the mercury in a thermometer moves 1 cm when the temperature changes by 1 °C, its sensitivity is 1 cm/°C (it is basically the slope dy/dx assuming a linear characteristic).

Sensor deviations.

Since sensors cannot replicate an ideal transfer function, several types of deviations can occur which limit sensor accuracy:

- Since the range of the output signal is always limited, the output signal will eventually reach a minimum or maximum when the measured property exceeds the limits. The full scale range defines the maximum and minimum values of the measured property
- The sensitivity may in practice differ from the value specified. This is called a sensitivity error. This is an error in the slope of a linear transfer function.
- If the output signal differs from the correct value by a constant, the sensor has an offset error or bias.
- Nonlinearity is deviation of a sensor's transfer function from a straight line transfer function. Usually, this is defined by the amount the output differs from ideal behavior over the full range of the sensor, often noted as a percentage of the full range.
- Deviation caused by rapid changes of the measured property over time is a dynamic error. Often, this behavior is described with a bode plot showing sensitivity error and phase shift as a function of the frequency of a periodic input signal.
- If the output signal slowly changes independent of the measured property, this is defined as drift. Long term drift over months or years is caused by physical changes in the sensor.

- Noise is a random deviation of the signal that varies in time.
- A hysteresis error causes the output value to vary depending on the previous input values. If a sensor's output is different depending on whether a specific input value was reached by increasing vs. decreasing the input, then the sensor has a hysteresis error.
- If the sensor has a digital output, the output is essentially an approximation of the measured property. This error is also called quantization error.
- If the signal is monitored digitally, the sampling frequency can cause a dynamic error, or if the input variable or added noise changes periodically at a frequency near a multiple of the sampling rate, aliasing errors may occur.
- The sensor may to some extent be sensitive to properties other than the property being measured. For example, most sensors are influenced by the temperature of their environment.

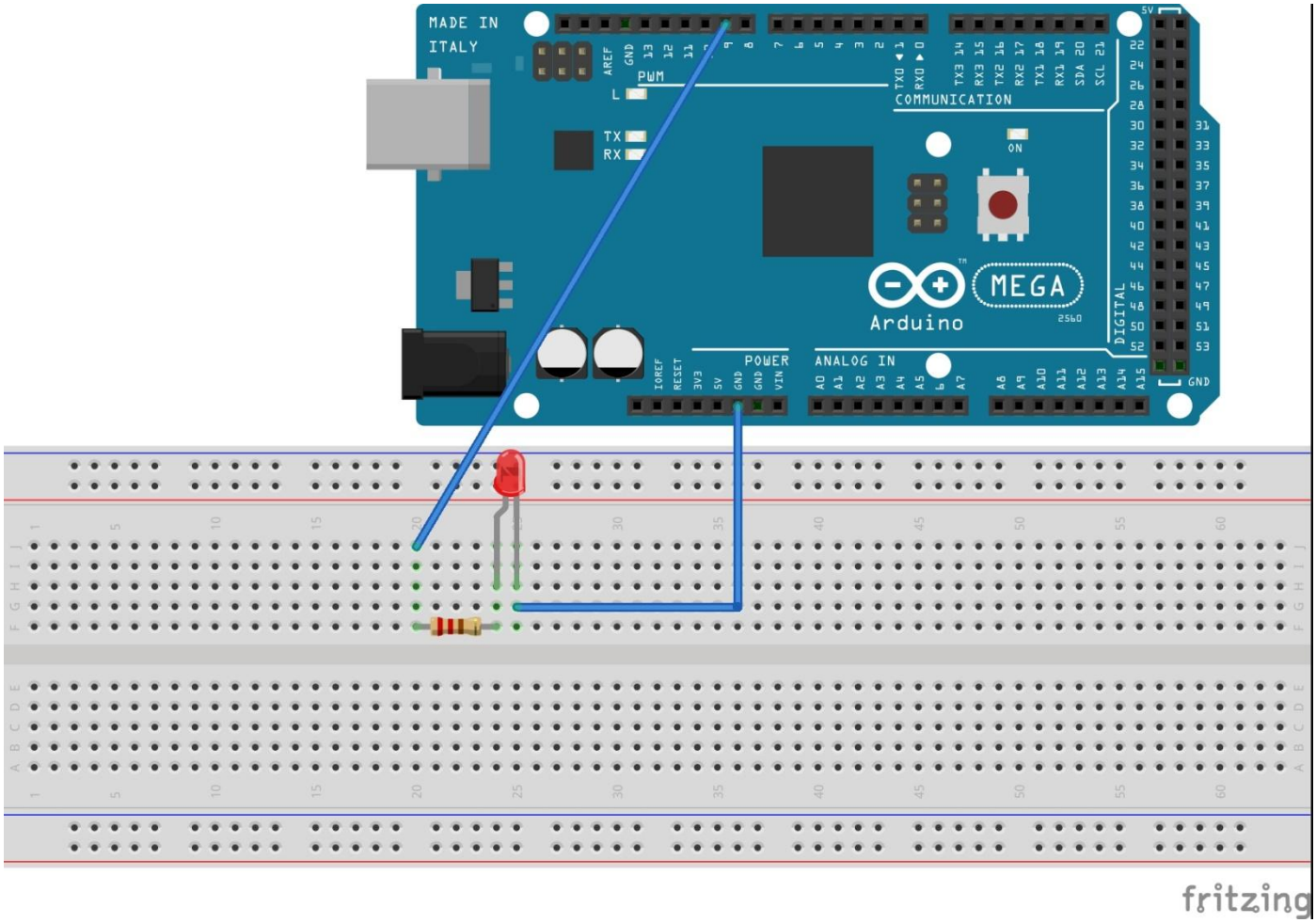
All these deviations can be classified as systematic errors or random errors. Systematic errors can sometimes be compensated for by means of some kind of calibration strategy. Noise is a random error that can be reduced by signal processing, such as filtering, usually at the expense of the dynamic behavior of the sensor.

Resolution

The *sensor resolution* or *measurement resolution* is the smallest change that can be detected in the quantity that it is being measured. The resolution of a sensor with a digital output is usually the numerical resolution of the digital output. The resolution is related to the precision with which the measurement is made, but they are not the same thing. A sensor's accuracy may be considerably worse than its resolution.

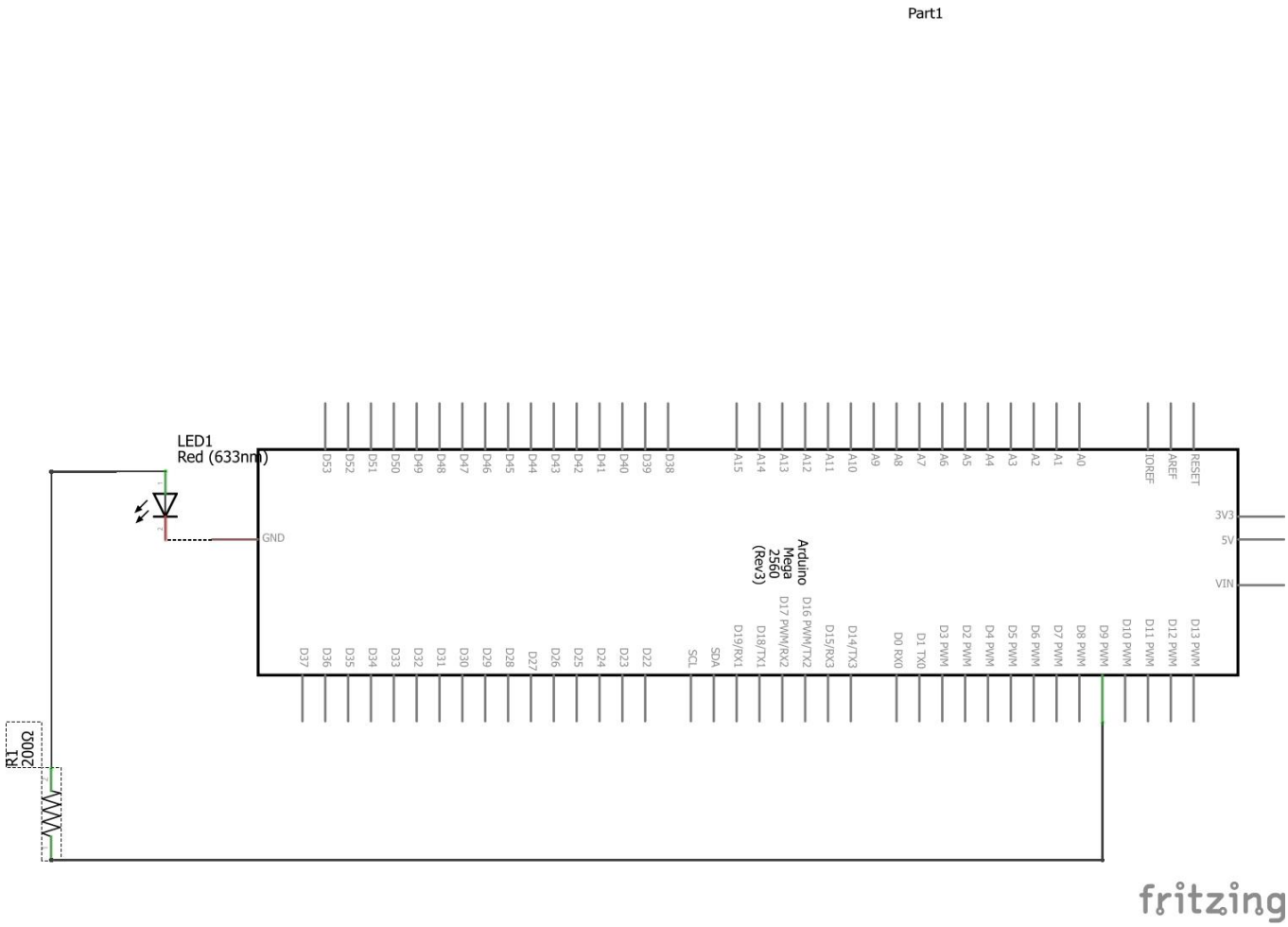
What type of sensors we use is the sensors that convert physical parameters to electrical signals

Arduino blink and the Fade



Bread board connection

Schematic diagram



Understanding the circuit

From LED data sheet maximum LED current = 30 mA

Optimum = 20 mA = 0.02 A

Voltage = 5V

Forward Voltage drop across LED =2 V

Voltage across resistor = 5 – 2 = 3 V (Kirchhoff voltage law)

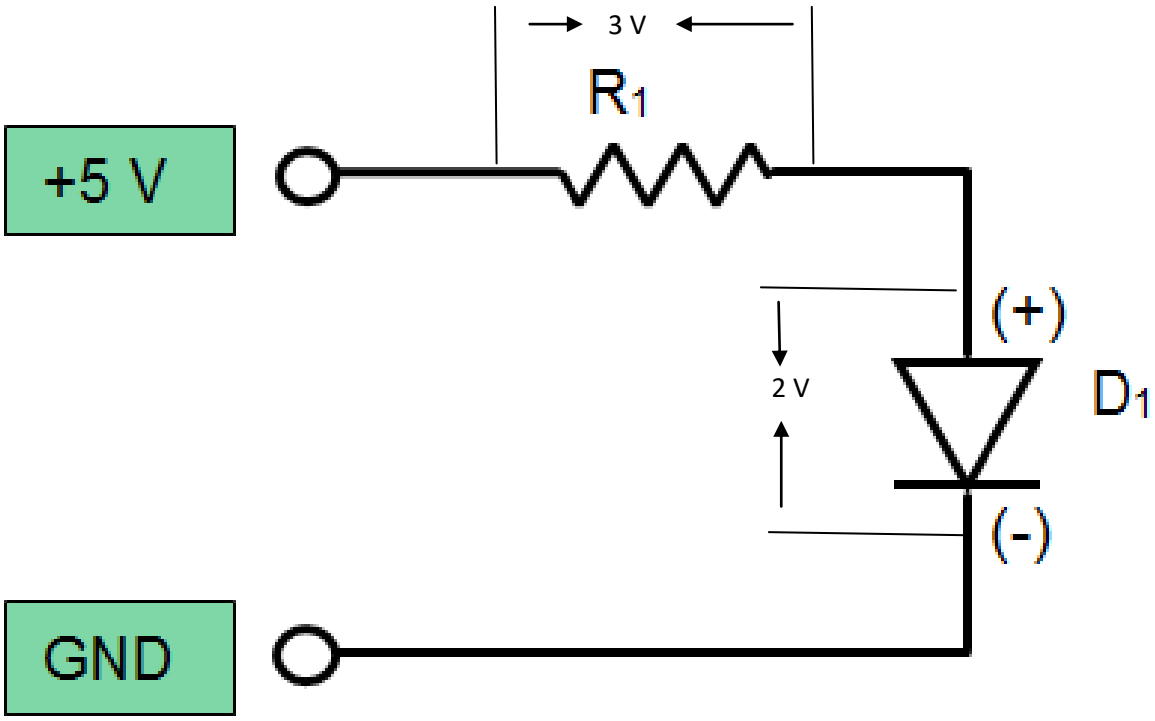
We know by ohms law V = I * R

So our resistor should be R = 3 / 0.02 = 150 ohm

We know the Power P = V * I

So resistor power drop is 3 V * 0.02 A = 0.06 W

So we can choose a 0.25 W resistor



Blink code

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}
```

```
// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

Fade

```
 // Variable Declaration
int led = 9;          // the PWM pin the LED is attached to
int brightness = 0;    // how bright the LED is
int fadeAmount = 5;    // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup() {
  // declare pin 9 to be an output:
  pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
  // set the brightness of pin 9:
  analogWrite(led, brightness);

  // change the brightness for next time through the loop:
  brightness = brightness + fadeAmount;

  // reverse the direction of the fading at the ends of the fade:
  if (brightness <= 0 || brightness >= 255) {
    fadeAmount = -fadeAmount;
```

```
  }
  // wait for 30 milliseconds to see the dimming effect
  delay(30);
}
```

Example code for library function

```
/*
 LiquidCrystal Library - Hello World

Demonstrates the use a 16x2 LCD display.  The LiquidCrystal
library works with all LCD displays that are compatible with the
Hitachi HD44780 driver. There are many of them out there, and you
can usually tell them by the 16-pin interface.

This sketch prints "Hello World!" to the LCD
and shows the time.

 The circuit:
* LCD RS pin to digital pin 12
* LCD Enable pin to digital pin 11
* LCD D4 pin to digital pin 5
* LCD D5 pin to digital pin 4
* LCD D6 pin to digital pin 3
* LCD D7 pin to digital pin 2
* LCD R/W pin to ground
* LCD VSS pin to ground
* LCD VCC pin to 5V
* 10K resistor:
```

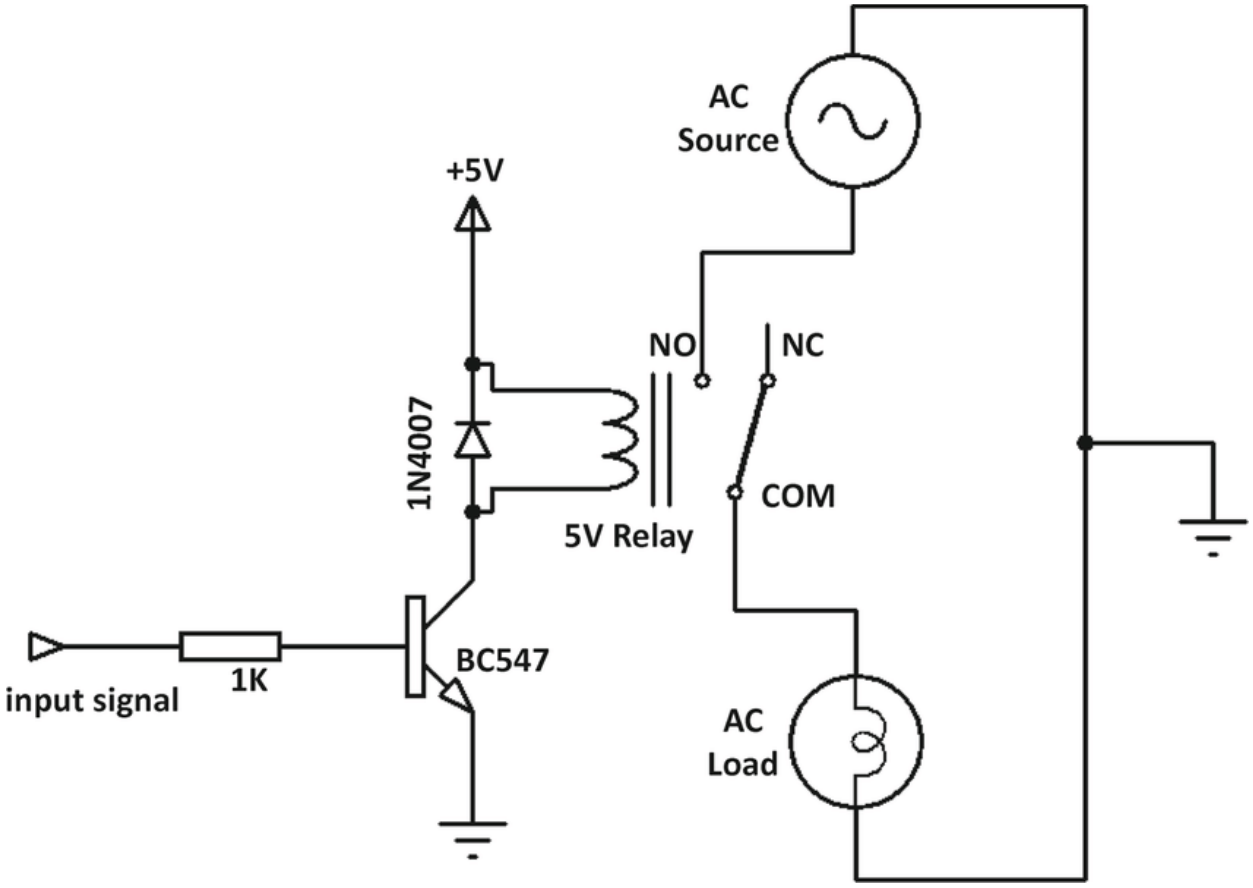
```
 * ends to +5V and ground
 * wiper to LCD VO pin (pin 3)


*/

// include the library code:
#include <LiquidCrystal.h>

// initialize the library by associating any needed LCD interface pin
// with the arduino pin number it is connected to
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup() {
  // set up the LCD's number of columns and rows:
  lcd.begin(16, 2);
  // Print a message to the LCD.
  lcd.print("hello, world!");
}

void loop() {
  // set the cursor to column 0, line 1
  // (note: line 1 is the second row, since counting begins with 0):
  lcd.setCursor(0, 1);
```

```
  // print the number of seconds since reset:
  lcd.print(millis() / 1000);
}
```

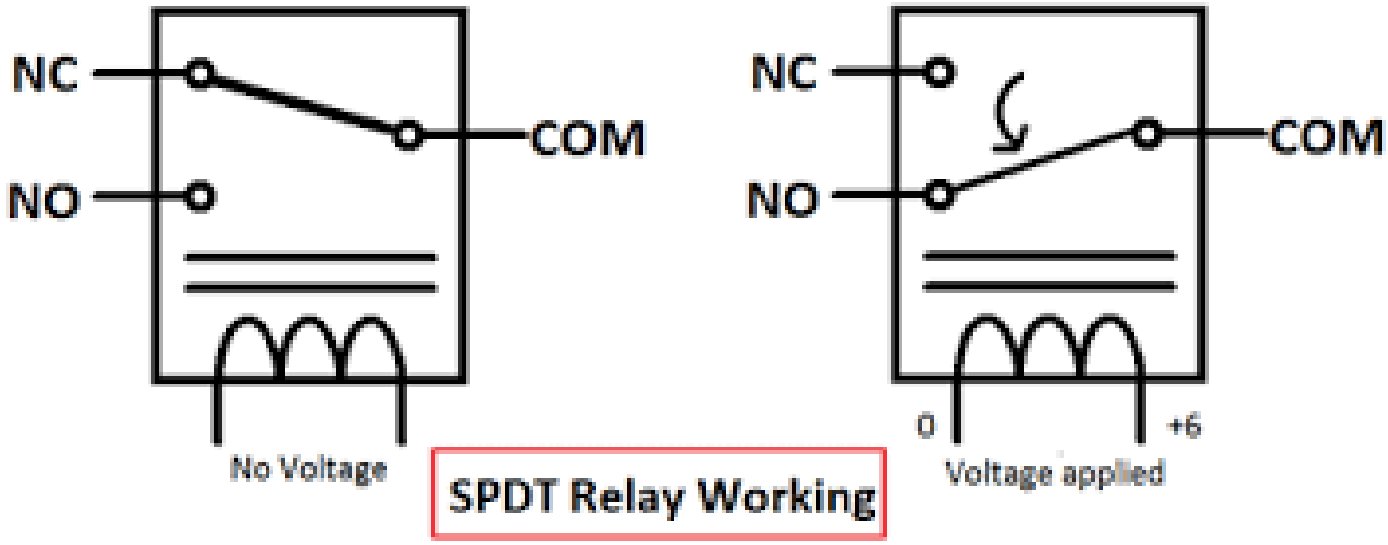
Evive Blink

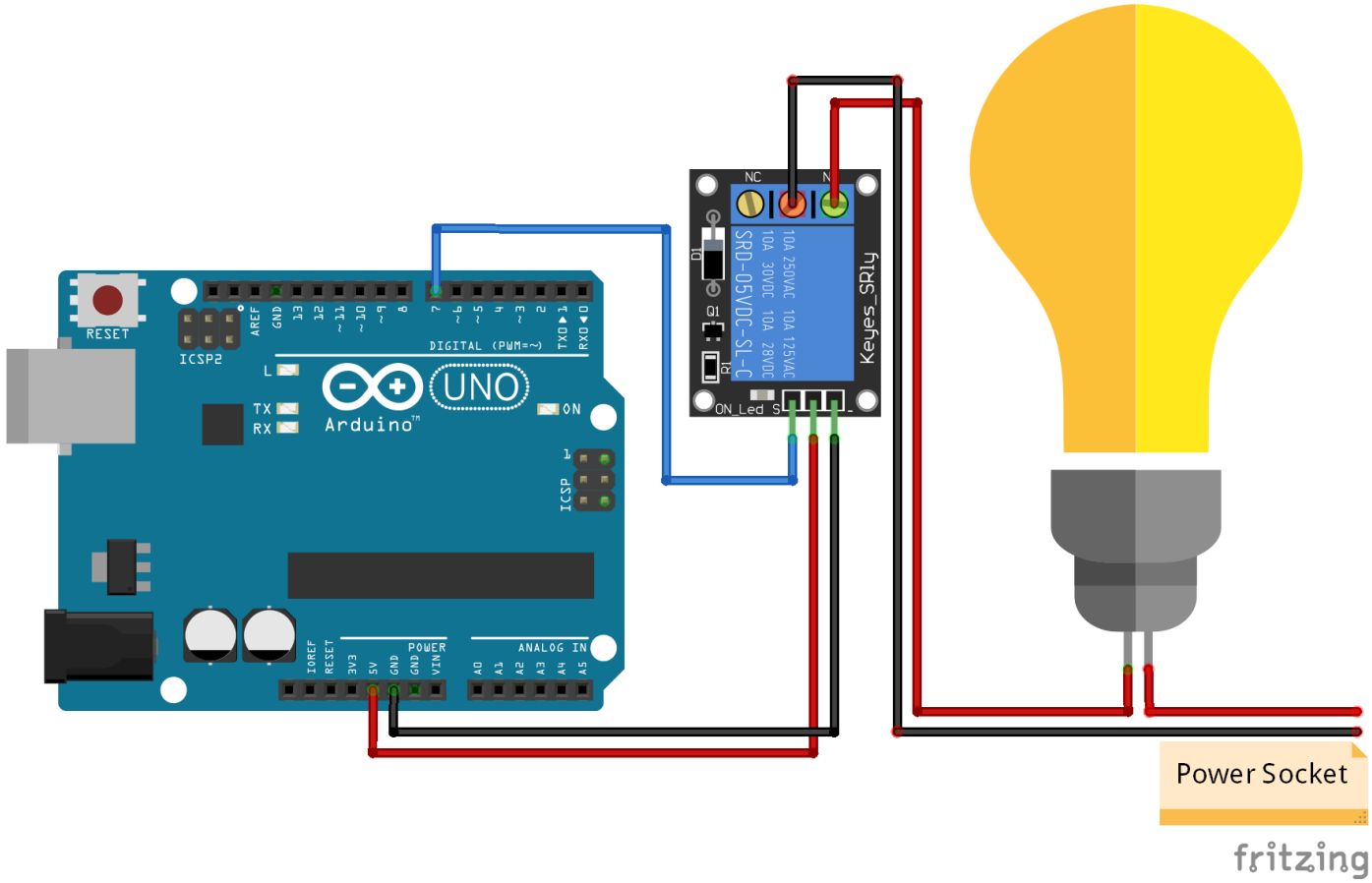
```
//This c++ code is generated by PictoBlox

#include <evive.h>

void setup() {
	//put your setup code here, to run once:
	pinMode(13, OUTPUT);

}

void loop() {
	//put your main code here, to run repeatedly:

	digitalWrite(13, true);
	delay(1 * 1000);
	digitalWrite(13, false);
	delay(1 * 1000);
}
```

Evive, Dabble and pictoblox Light on off



Relay driver circuit

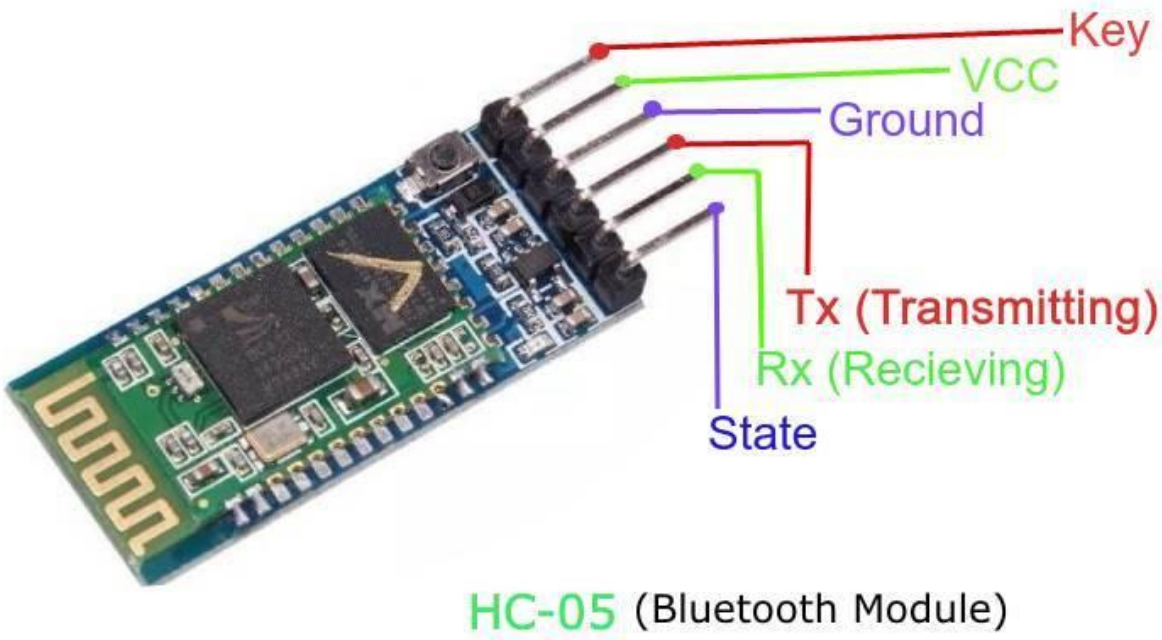

NC
NO
COM
No Voltage
NC
NO
COM
0
+6
Voltage applied
SPDT Relay Working


RESET
ICSP2
DIGITAL (PWM=~)
UNO
Arduino
POWER
ANALOG IN
ICSP
NC
SRD-05VDC-SL-C
Keyes_SRly
ON_Led
Power Socket
fritzing

HC-05 pinout, specifications, datasheet and HC05 Evive connection

HC-05 Bluetooth module is a Bluetooth to serial converter that connects microcontrollers (like Arduino) to other Bluetooth enabled devices. HC-05 pinout and specifications are given below.

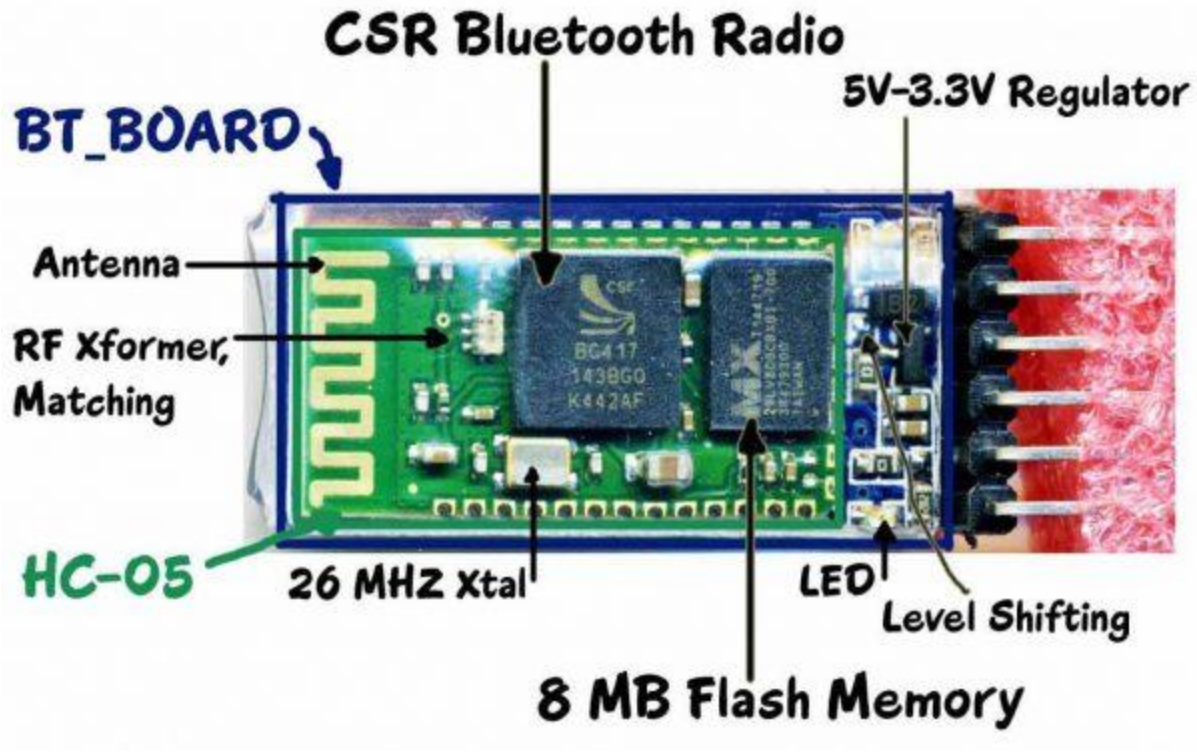
HC-05 Pinout with Description:



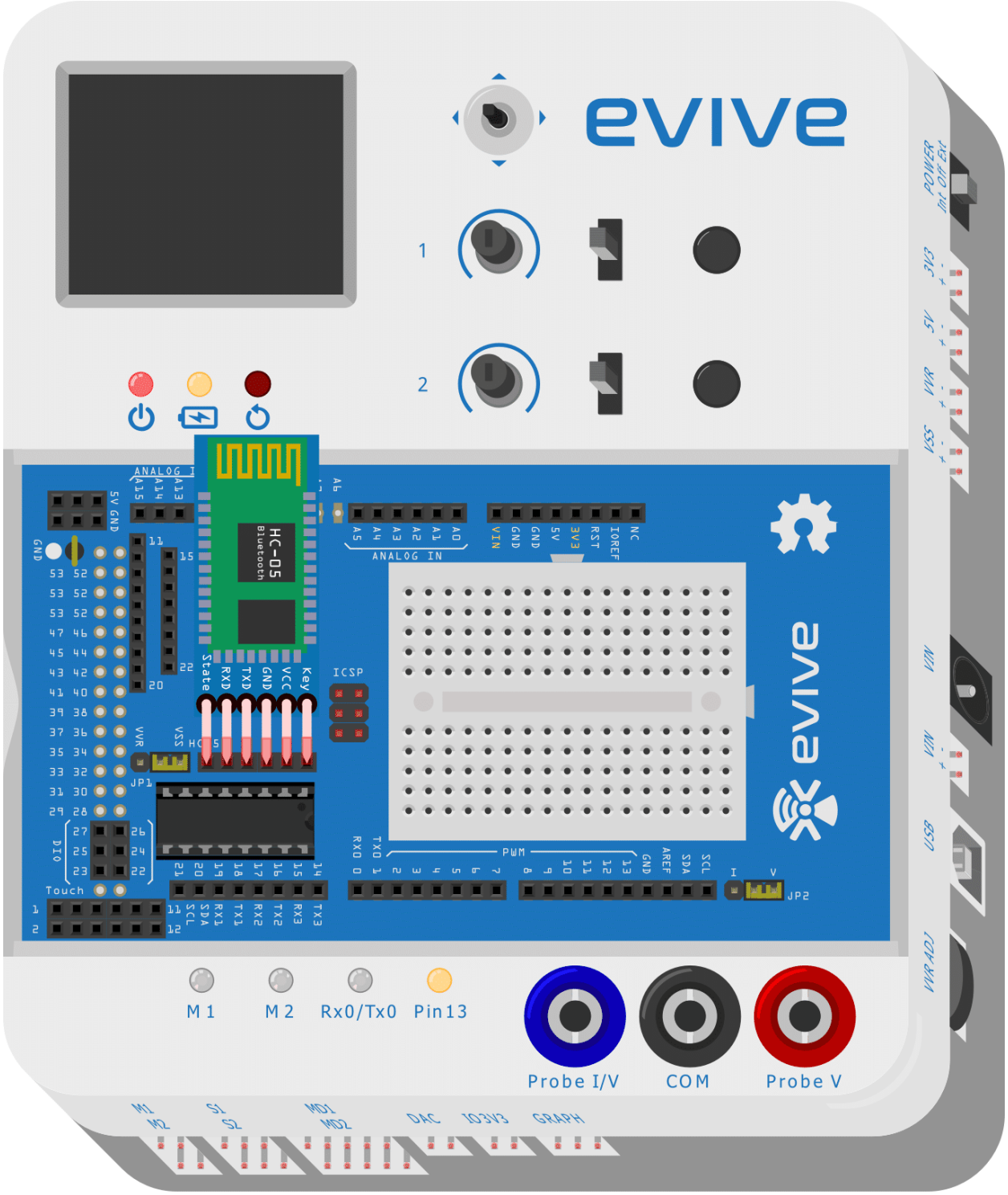
PIN NO.	Pin Name	Pin Description
1.	**KEY/En**	This pin is used to bring the Bluetooth module in AT commands mode. By default, this pin operates in data mode. The Key/EN pin should be high to operate Bluetooth in command mode. In HC-05, the default baud speed in command mode is 38400bps and 9600 in data mode.

2.	**VCC**	Used to power the Bluetooth module. Give 5V / 3.3 V to this Pin.
3.	**GND**	The ground pin of the module
4.	**TXD**	Connect this pin with the RXD pin of the Microcontroller. This pin transmits Serial data (wireless signals received by the Bluetooth module are converted by module and transmitted out serially on this pin)
5.	**RXD**	Connect this pin to the TXD pin of the Microcontroller. The HC-05 Bluetooth module receives the data from this pin and then transmits it wirelessly.
6.	**STATE**	It is used to check if the module is connected or not. It acts as a status indicator.

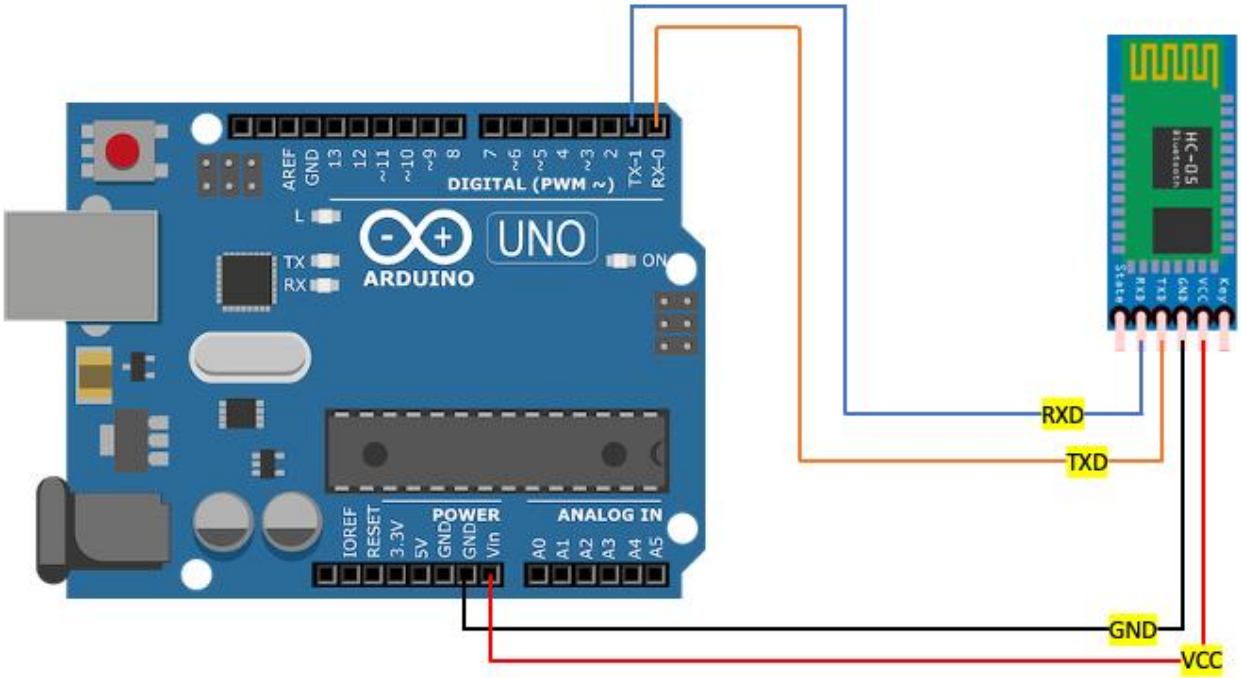
HC-05 Specifications



- Bluetooth protocol: Bluetooth Specification **v2.0+EDR** (Enhanced Data Rate)
- Frequency: **2.4GHz ISM band**
- Modulation: **GFSK** (Gaussian Frequency Shift Keying)
- Emission power: ≤4dBm, Class 2
- Sensitivity: ≤-84dBm at 0.1% BER
- Speed: Asynchronous communication: **2.1Mbps (Max) / 160 kbps**, Synchronous communication: **1Mbps/1Mbps**
- Security: Authentication and encryption
- Profiles: Bluetooth serial port
- Supply Voltage: **+3.3V to 6.0 V**
- Supply Current: **30mA**
- Working temperature: -20 ~ +75Centigrade
- Dimension: **26.9mm x 13mm x 2.2 mm**
- HC-05 Bluetooth module follows the IEEE 802.15.1 standardized protocol, through which one can build a wireless Personal Area Network (PAN). It uses frequency-hopping spread spectrum (FHSS) radio technology to send data over the air.

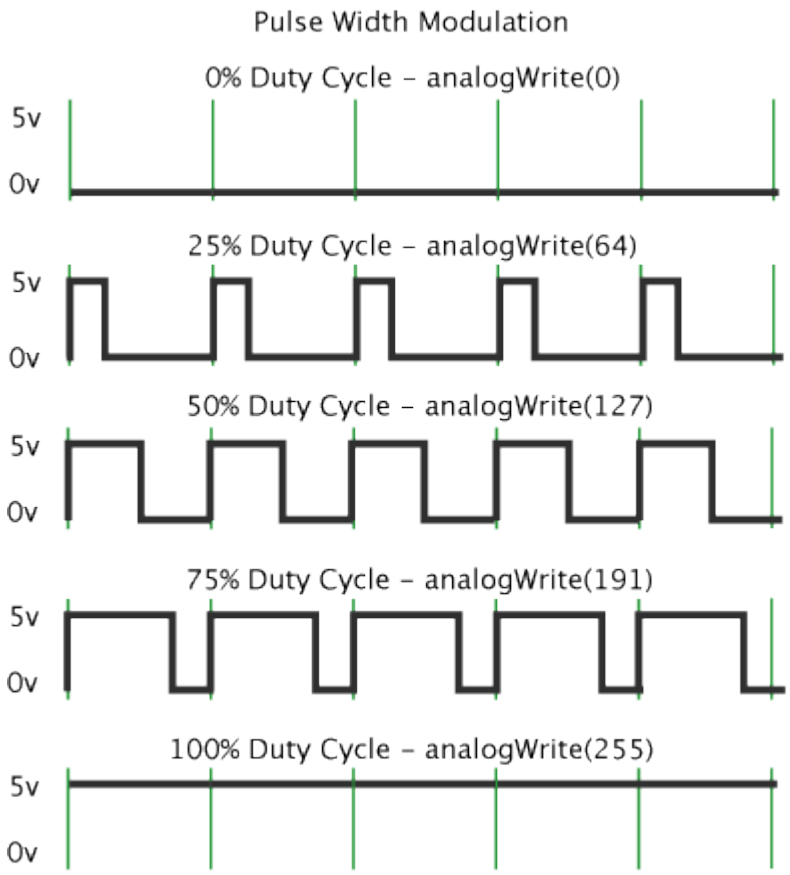

evive
1
2
M 1
M 2
Rx0/Tx0
Pin13
Probe I/V
COM
Probe V
HC-05
Bluetooth
ANALOG IN
ICSP
PWM
Touch
JP1
JP2
POWER
ON OFF EXT
3V3
5V
VAR
VSS
VIN
USB
VAR ADJ
M1
M2
S1
S2
MD1
MD2
DAC
IO3V3
GRAPH
fritzing

evive
M1
M2
Rx0/Tx0
PIN13
COM
PROBE/V
GRAPH
PROBE1/V
IO3V3
DAC
MD1
MD2
S1
S2
M1
M2

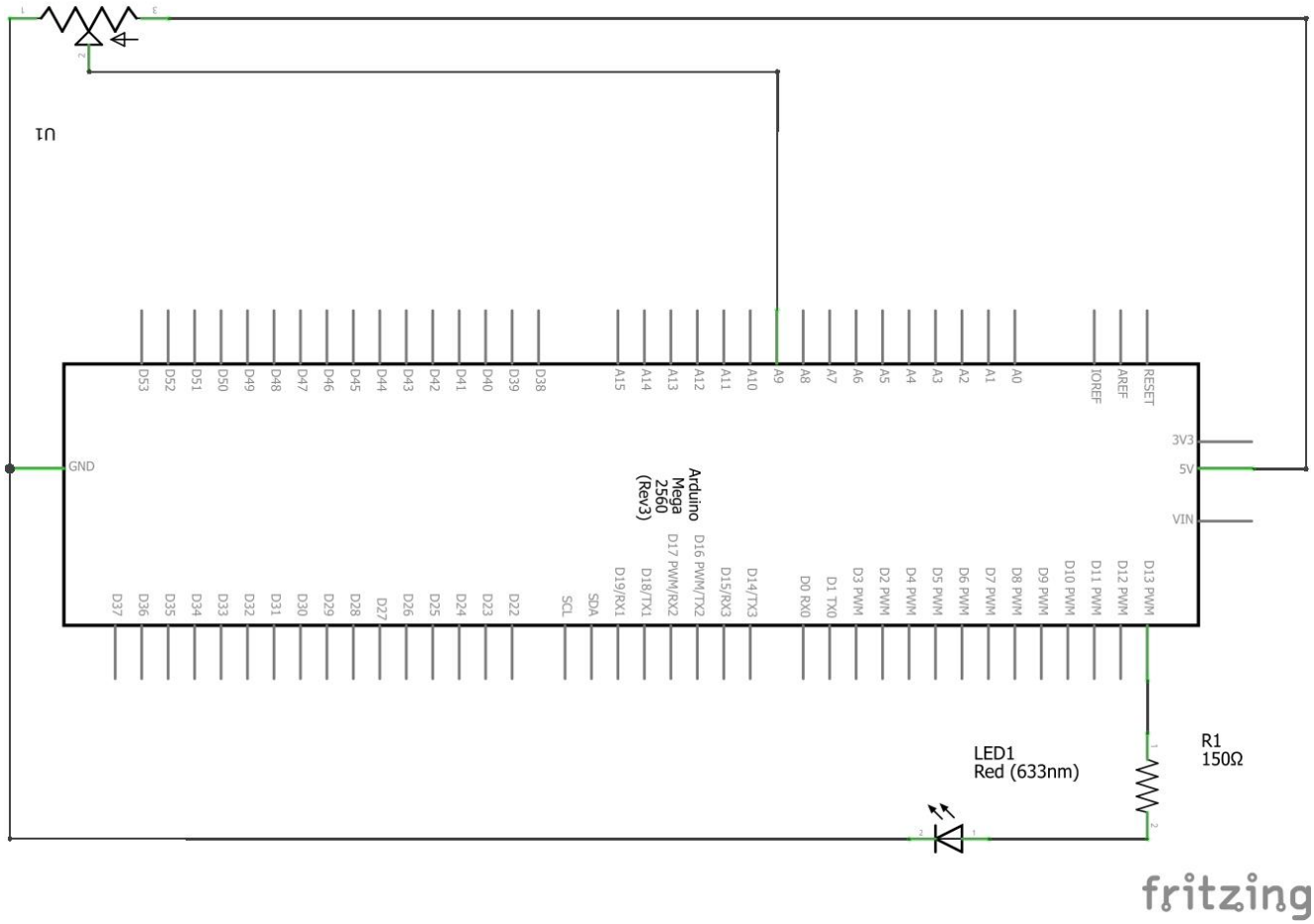

AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX-1
RX-0
DIGITAL (PWM ~)
L
TX
RX
ARDUINO
UNO
ON
IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
HC-05
Bluetooth
RXD
TXD
GND
VCC

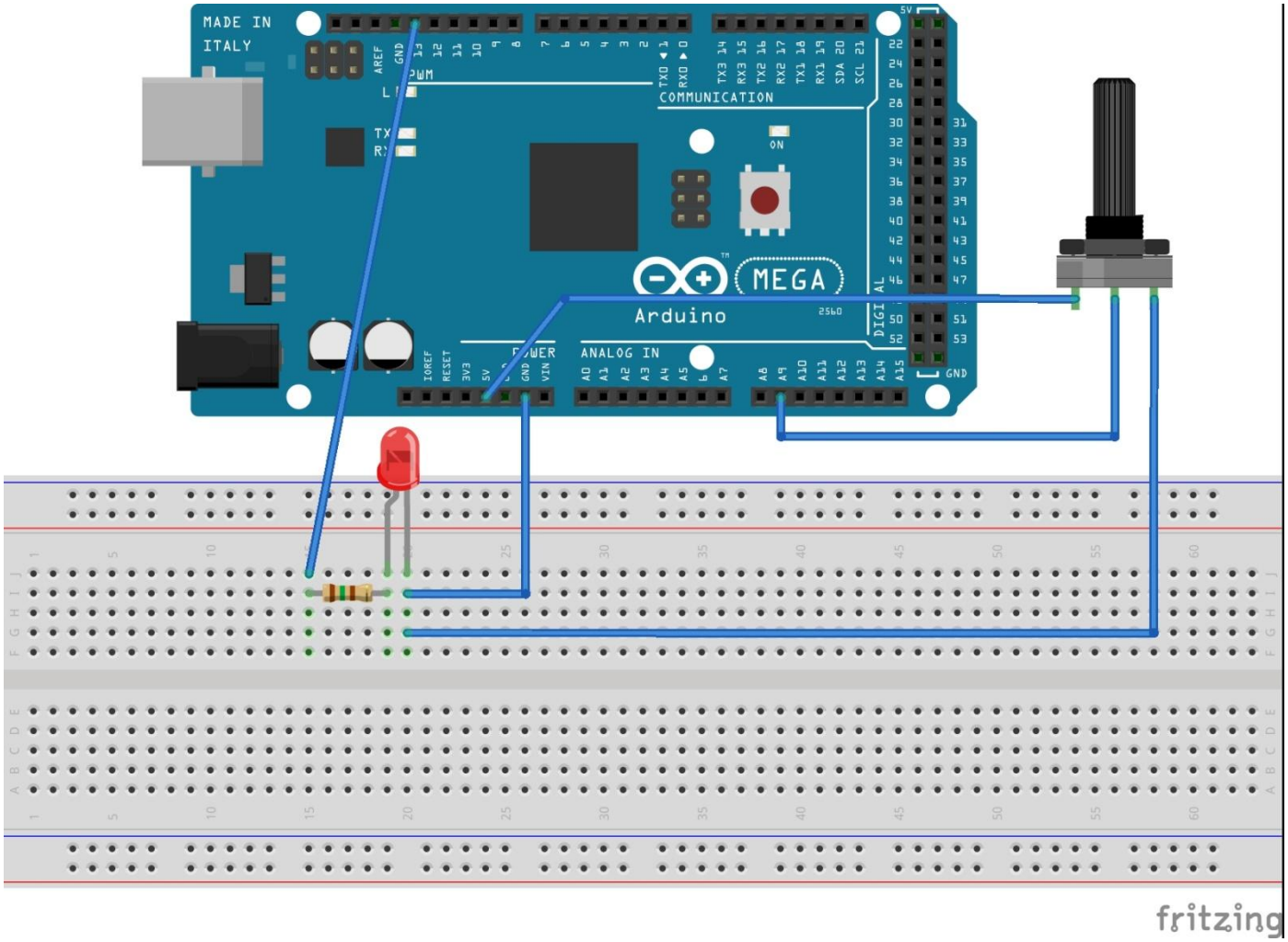
```
//This c++ code is generated by PictoBlox
#include <evive.h>

//Included Libraries
#include <Dabble.h>

void setup() {
	//put your setup code here, to run once:
	Dabble.begin(115200);
	pinMode(13, OUTPUT);

}

void loop() {
	//put your main code here, to run repeatedly:
	Dabble.processInput();

	if(Terminal.compareString("light on")) {
		digitalWrite(13, false);
	}
	if(Terminal.compareString("light off")) {
		digitalWrite(13, true);
	}
}
```

Pulse width modulation



Part1


U1
Arduino
Mega
2560
(Rev3)
GND
D53
D52
D51
D50
D49
D48
D47
D46
D45
D44
D43
D42
D41
D40
D39
D38
A15
A14
A13
A12
A11
A10
A9
A8
A7
A6
A5
A4
A3
A2
A1
A0
IOREF
AREF
RESET
3V3
5V
VIN
D37
D36
D35
D34
D33
D32
D31
D30
D29
D28
D27
D26
D25
D24
D23
D22
SCL
SDA
D19/RX1
D18/TX1
D17 PWM/RX2
D16 PWM/TX2
D15/RX3
D14/TX3
D0 RX0
D1 TX0
D3 PWM
D2 PWM
D4 PWM
D5 PWM
D6 PWM
D7 PWM
D8 PWM
D9 PWM
D10 PWM
D11 PWM
D12 PWM
D13 PWM
LED1
Red (633nm)
R1
150Ω
fritzing


MADE IN
ITALY
COMMUNICATION
MEGA
Arduino
POWER
ANALOG IN
fritzing

```
//This c++ code is generated by PictoBlox

#include <evive.h>

void setup() {
	//put your setup code here, to run once:
	pinMode(9, INPUT);
	pinMode(13, OUTPUT);



}

void loop() {
	//put your main code here, to run repeatedly:



	analogWrite(13, (analogRead(9) / 3));
}
```

Basics of Electrical and Electronics

Voltage

According to structure of an atom, we know that there are two types of charges: positive and negative. A force of attraction exists between these positive and negative charges. Certain amount energy (work) is required to overcome the force and move the charges to a specified distance. All opposite charges possess a certain amount of potential energy because of separation between them. The difference in potential energy of charges is called potential difference.

Potential difference in electrical terminology is known as **voltage**, and is denoted by **V**. It is denoted by watt (W) per unit charge (Q)

i.e. $V = \frac{W}{Q}$

or $V = \frac{dw}{dq}$

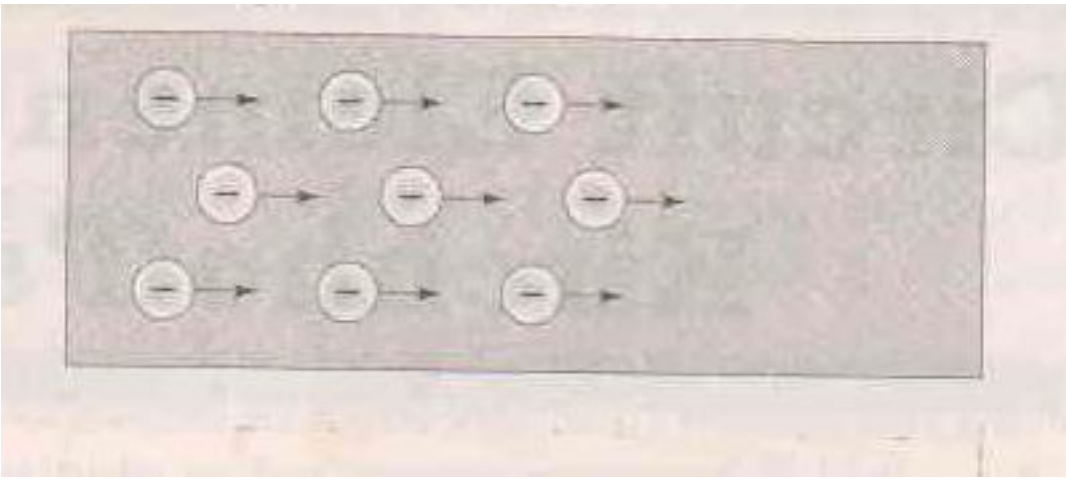
where dw is the small change in energy

and dq is the small change in charge.

Where energy (W) is expressed in joules (J), charge (Q) in coulombs (C), and voltage in (V). One volt is the potential difference between two points when one joule of energy is used to pass one coulomb of charge from one point to another.

Current

There are free electrons available in all semiconductor and conductive materials. These free electrons move random in all directions within the structure in the absence of external pressure or voltage. If a certain amount of pressure is applied across the material, all the free electrons move in one direction depeding on the polarity of the applied voltage as shown in the figure.



The movement of electrons from one end of the material to other end of the material constitute an electrical current denoted by **I.** The conventional direction of current flow is opposite to the flow of – ve charges
i.e. the electrons.

Current is defined as the rate of flow of electrons in a conductive or semi conductive material. It is measured by the number of electrons that flow past a point in a unit time.

Expressed mathematically $I = \frac{Q}{t}$

Where I is the current, Q is the charge of electrons and t is the time or

$$I = \frac{dq}{dt}$$

where dq is small change in charge of electrons and dt is small change in the time.

In practice unit ampere is used to measure current, denoted by A. One ampere is equal to one coulomb per second. One coulomb is the charge carried by $6.25 * 10^{18}$ electrons.

Power and Energy

Energy is the capacity for doing work. i.e. energy is nothing but stored work. Energy may exist in so many forms such as mechanical, chemical, electrical and so on. Power is the rate of change of energy, and is denoted by P. If certain amount of energy is used over a certain length of time then

$$P = \frac{\text{energy}}{\text{time}} = \frac{\text{W}}{\text{t}}$$

$$P = \frac{dw}{dt} = \frac{dw}{dq} * \frac{dq}{dt}$$

$$= V * I = \mathbf{VI} \ \text{W}$$

Energy is measured in joules (J), time in seconds (s), and power in watts (W).

By definition one watt is the amount of power generated when one joule of energy is consumed in one second. Thus no joules consumed in one second is always equal to number of watts.